ANUBIS: Identity, IAM and Tracking for AI Agents

Full project blueprint: name, architecture, data model, flows, build plan, and demo.

1. The Idea in One Paragraph

Autonomous AI agents call tools, touch data, and delegate to other agents, but today most of them run on shared API keys with no identity, no scoped permissions, and no reliable record of what they did. **ANUBIS** is an identity and control plane that sits between agents and the resources they use. Every agent registers and gets a verifiable identity. Every action goes through an enforcement gateway that checks scoped, short-lived credentials and policy. Every decision is written to a tamper-evident audit trail, and any agent can be revoked instantly.

2. Project Name

ANUBIS stands for *Agent Namespace, Unified Boundaries, Identity and Surveillance*. Anubis is the Egyptian god who weighs who may pass into the underworld, which fits the core job: every agent is checked before it is let through. The name is short and works as a repo name, CLI name, and domain.

Other names considered: **Charon**, **Hecate**, **Talos**, **Heimdall**.

Tagline: *Every agent is weighed before it passes, and the record never lies.*

3. Goals and Non-Goals

Goals

- Unique, verifiable identity for every agent and every running instance.
- Least-privilege, short-lived, task-scoped access.
- Delegation tracking: who authorized this action, through which chain of agents.
- Complete, tamper-evident audit trail with trace IDs across multi-agent workflows.
- Detection of abnormal behavior and instant revocation.
- Works with agents without changing their code (proxy), with optional SDK for richer context.

Non-goals (for v1)

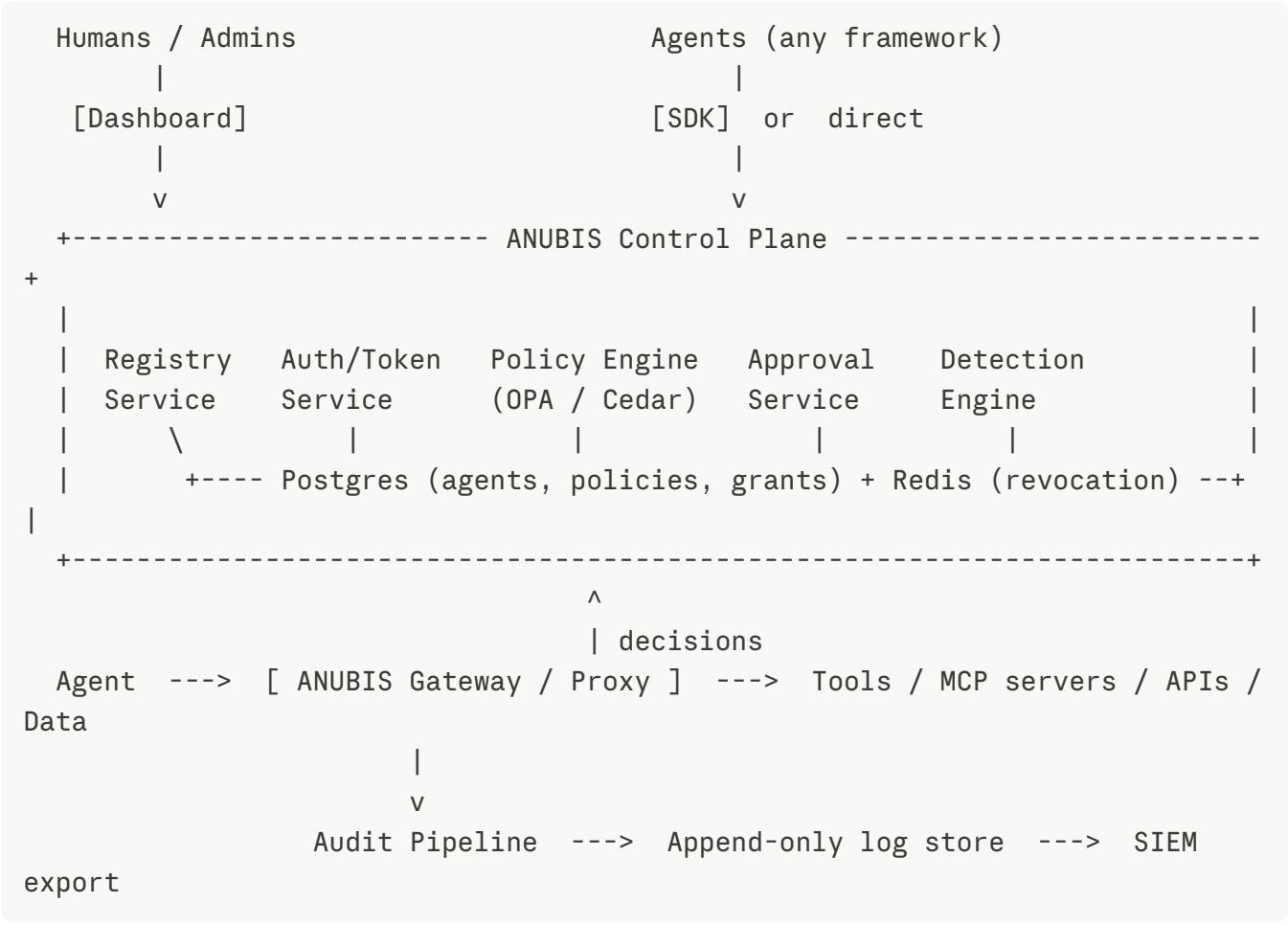
- Building agents themselves.
- Replacing enterprise IAM such as Okta or Entra. ANUBIS complements them by federating human identity.
- Solving prompt injection. ANUBIS limits the blast radius when it happens.

4. Threat Model

Design against these threats, since they justify every component.

1. **Credential theft or sharing**: a leaked static key gives unlimited access. Mitigation: short-lived tokens bound to an agent instance.
2. **Confused deputy**: an agent with broad rights is tricked into acting for the wrong party. Mitigation: delegation chains and on-behalf-of scoping.
3. **Privilege escalation**: an agent or sub-agent obtains more rights than its parent. Mitigation: child scope must be a subset of parent scope.
4. **Prompt-injection-driven misuse**: injected text makes an agent call dangerous tools. Mitigation: policy on tool and arguments, approval gates, rate limits.
5. **Data exfiltration**: bulk reads or sends to external destinations. Mitigation: volume limits, destination allowlists, anomaly detection.
6. **Rogue or forgotten agents**: unknown agents still holding access. Mitigation: mandatory registry, expiry, inventory.
7. **Log tampering**: an attacker covers tracks. Mitigation: append-only, hash-chained logs.
8. **Replay of tokens**: Mitigation: short TTL, unique token IDs (jti), instance binding.

5. High-Level Architecture



Two planes. The *control plane* manages identities, policies, and approvals. The *data plane* (the gateway) enforces them on every call. Keeping them separate lets the gateway stay fast and the control plane stay simple.

6. Components in Detail

6.1 Agent Registry

- Stores agents as first-class identities: ID, name, owner, purpose, framework, environment, risk tier, status (active, suspended, revoked), expiry.
- Registration requires an owner (human or team) so every agent is accountable.
- Each agent has a keypair. The public key is stored, and the private key stays with the agent runtime.
- Inventory views: all agents, unowned agents, expired agents, high-risk agents.

6.2 Auth and Token Service

- **Agent authentication**: the agent proves possession of its key (signed challenge) or presents a platform attestation, for example a Kubernetes service account token.
- **Token issuance**: signed JWT with claims: `sub` (agent ID), `instance_id`, `act` (delegation chain), `on_behalf_of` (human), `scope`, `task_id`, `jti`, `exp` (minutes, not hours).
- **Token exchange** (OAuth 2.0 style): an agent exchanges its token for a narrower one, either for a sub-agent or for a specific tool.
- **Rule**: a derived token's scope must be a subset of the parent's.
- Signing keys rotate on a schedule, published via a JWKS endpoint.

6.3 Policy Engine

- Policy as code using **OPA (Rego)** or **Cedar**. Start with OPA since it is widely documented.
- Decision input: agent identity, owner, risk tier, delegation chain, tool name, arguments, resource, time, recent activity counters.
- Decision output: `allow`, `deny`, or `require_approval`, plus a reason string.
- Example policies: finance agents can read but never write ledger tools; any email to an external domain requires approval; a sub-agent cannot call tools its parent cannot.

6.4 Gateway / Proxy (Enforcement Point)

- Primary integration: an **MCP proxy**. Agents point at ANUBIS instead of the real MCP server. ANUBIS forwards allowed calls and blocks the rest.
- Secondary: an HTTP forward proxy for REST tools.
- Per request: validate the token (signature, expiry, revocation list), check scope, call the policy engine, apply rate limits, redact secrets in logs, forward, and record the outcome.
- Fail closed: if the policy engine is unreachable, deny.
- Credential brokering: the gateway holds the real downstream secrets, so agents never see them.

6.5 Approval Service

- For `require_approval` decisions, the gateway pauses the call and creates an approval request.

- Approvers respond through the dashboard, Slack, or email. Approvals are time-limited and single-use.
- Everything is logged, including who approved.

6.6 Audit Pipeline

- Every event is a structured record: timestamp, trace ID, agent ID, instance, delegation chain, human principal, tool, redacted arguments, policy decision, reason, latency, outcome.
- **Hash chain**: each record stores the hash of the previous record, so edits and deletions are detectable. Periodically anchor the latest hash somewhere external, such as a signed checkpoint file or a public gist.
- Storage: Postgres for v1, with an option to move to ClickHouse or OpenSearch for scale.
- Export: syslog, JSON lines, or webhook to a SIEM.

6.7 Detection Engine

- **Rule-based first**: burst of denied calls, tool never used before, off-hours activity, scope probing, high data volume.
- **Baselines**: per-agent profile of normal tools and rates.
- **Optional ML later**: simple anomaly scoring such as isolation forest on per-agent features.
- Actions: alert, require approval for subsequent calls, suspend, or revoke.

6.8 Revocation

- Kill switch per agent, per instance, per task, or per human principal.
- Revoked token IDs and agent IDs go to Redis and the gateway checks them on every call. Tokens are short-lived anyway, so revocation takes effect within seconds.

6.9 Dashboard

- Agent inventory and status.
- Live activity feed with filters.
- **Delegation graph**: visualize human, agent, and sub-agent chains for a trace.
- Audit search and integrity verification button.
- Policy editor and simulator (test a request against policies).
- Approvals queue, alerts, and revoke buttons.

6.10 SDK (Optional Enrichment)

- Thin Python and TypeScript libraries: `register()`, `get_token()`, `delegate()`, `with_task(task_id, intent)`.
- Adds context the proxy cannot see, such as task intent and sub-agent spawning.
- Framework adapters for LangChain, LangGraph, and the Claude Agent SDK.

7. Data Model

agents: id, name, owner_id, purpose, framework, environment, risk_tier, public_key, status, created_at, expires_at

owners: id, name, email, team, external_idp_id

agent_instances: id, agent_id, started_at, ended_at, runtime_info, attestation

roles: id, name, description

grants: id, agent_id or role_id, resource_pattern, actions, conditions, expires_at

policies: id, name, version, rego_source, status, created_by, created_at

tokens: jti, agent_id, instance_id, parent_jti, scope, task_id, issued_at, expires_at, revoked_at

tasks: id, root_agent_id, on_behalf_of, intent, started_at, status

tools: id, name, server, risk_level, argument_schema, downstream_credential_ref

audit_events: id, seq, prev_hash, hash, timestamp, trace_id, task_id, agent_id, instance_id, chain, tool, args_redacted, decision, reason, outcome, latency_ms

approvals: id, audit_event_id, requested_at, approver_id, decision, decided_at, expires_at

alerts: id, agent_id, rule, severity, details, status, created_at

8. Key Flows

8.1 Registration

1. An owner registers an agent via the dashboard, API, or CLI with name, purpose, and risk tier.
2. ANUBIS stores the identity and the agent's public key.
3. Optional admin approval for high-risk tiers.

8.2 Starting a Task and Getting a Token

1. A user or system starts a task: `POST /tasks` with intent and requested scope.
2. The agent authenticates by signing a challenge.
3. ANUBIS issues a short-lived token carrying agent, instance, task, human principal, and scope.

8.3 Tool Call Through the Gateway

1. Agent sends a tool call to the gateway with its token.
2. Gateway validates the token and checks revocation.
3. Gateway asks the policy engine for a decision.
4. On `allow`, forward with brokered credentials. On `deny`, return an error. On `require_approval`, hold and notify.
5. Record the audit event either way.

8.4 Delegation to a Sub-Agent

1. The parent agent calls token exchange with the child's ID and a requested scope.
2. ANUBIS checks child scope is a subset of the parent's and the child is registered.
3. The new token's `act` claim extends the chain, so every later call shows the full lineage.

8.5 Detection and Revocation

1. Detection sees an anomaly, such as 30 denied calls in a minute.
2. It raises an alert and, per policy, suspends the agent.
3. Redis revocation takes effect at the gateway. Subsequent calls are blocked.
4. The audit trail shows the full trace for the investigation.

9. Technology Stack

- **Backend**: Python with FastAPI (fast to build, good fit for your ML and security background). Go is an option later for the gateway.
- **Database**: PostgreSQL. **Cache and revocation list**: Redis.
- **Policy**: Open Policy Agent (as a sidecar or library).
- **Tokens**: JWT with ES256 or EdDSA, JWKS endpoint, `authlib` or `pyjwt`.

- **Gateway**: FastAPI or a small async proxy using `httpx`, implementing the MCP protocol for tool calls.
- **Frontend**: React with Vite, Tailwind, and a graph library such as React Flow or Cytoscape for delegation graphs.
- **Deployment**: Docker Compose for the demo, with Helm charts as a later stretch.
- **Observability**: OpenTelemetry for traces, Prometheus and Grafana for metrics.
- **Testing**: pytest, Locust or k6 for load, and an attack-scenario test suite.

10. API Surface (v1)

- `POST /agents`, `GET /agents`, `PATCH /agents/{id}` (suspend, revoke)
- `POST /auth/challenge`, `POST /auth/token`
- `POST /auth/exchange` (delegation and narrowing)
- `POST /tasks`, `GET /tasks/{id}`
- `GET /.well-known/jwks.json`
- `POST /policies`, `POST /policies/simulate`
- `GET /audit`, `GET /audit/verify`
- `GET /approvals`, `POST /approvals/{id}/decision`
- `GET /alerts`
- Gateway: `POST /mcp/{server}` (proxied tool calls)

11. Suggested Repository Structure

```
anubis/
  services/
    registry/        # agent identities and owners
    auth/            # token issuance, exchange, JWKS
    policy/          # OPA integration and policies/*.rego
    gateway/         # MCP and HTTP enforcement proxy
    audit/           # hash-chained event store
    detection/       # rules and baselines
    approvals/
  sdk/
    python/
    typescript/
  dashboard/         # React app
  demo/
    agents/          # sample agents and a malicious scenario
    mock-tools/      # fake email, db, and file MCP servers
  deploy/            # docker-compose, helm
  docs/              # architecture, threat model, API
  tests/
```

Tip: start as a **modular monolith** (one FastAPI app with clean module boundaries) and split into services only if needed. It is much faster to build and easier to demo.

12. Step-by-Step Build Plan

Timings assume focused part-time work. Adjust to your deadline.

Phase 0: Foundations (2 to 3 days)

1. Write a one-page problem statement and the threat model above.
2. Create the repo, set up Docker Compose with Postgres, Redis, and OPA.
3. Pick the project name, README skeleton, and license.

Phase 1: Identity (about 1 week)

1. Build the registry: agents and owners tables, CRUD API.
2. Generate keypairs for agents, store public keys.

3. Implement the challenge-response login and the token service (JWT, JWKS, key rotation).
4. Tests: valid, expired, tampered, and replayed tokens.

Phase 2: Gateway and Authorization (1 to 2 weeks)

1. Build the gateway that validates tokens and forwards to a mock MCP server.
2. Integrate OPA: write the first policies (tool allowlists, argument constraints).
3. Add the `deny` and `allow` paths with proper error responses and fail-closed behavior.
4. Add credential brokering so agents never see downstream secrets.
5. Tests: unauthorized tool calls, argument violations, policy outage.

Phase 3: Audit Trail (about 1 week)

1. Define the audit event schema and write events from the gateway.
2. Implement hash chaining and the `/audit/verify` endpoint.
3. Add secret redaction for arguments and results.
4. Test tampering: edit or delete a row and confirm verification fails.

Phase 4: Delegation and Tasks (about 1 week)

1. Add tasks, `on_behalf_of`, and trace IDs.
2. Implement token exchange with the scope-subset rule and the `act` chain.
3. Add a demo multi-agent workflow (planner, researcher, writer).
4. Test escalation attempts: child asks for more scope than its parent.

Phase 5: Approvals and Revocation (about 1 week)

1. Add the `require_approval` path with hold and resume.
2. Build the approval queue API, with Slack or email notification as a stretch.
3. Implement revocation lists in Redis and kill switches by agent, instance, and task.
4. Test that revoked agents are blocked within seconds.

Phase 6: Detection (about 1 week)

1. Implement rule-based detections: denial bursts, new tool use, off-hours, volume.
2. Add per-agent baselines.

3. Wire detections to automatic actions: alert, approval escalation, suspend.
4. Optional: simple anomaly scoring model.

Phase 7: Dashboard (1 to 2 weeks)

1. Agent inventory and status views.
2. Live activity feed.
3. Delegation graph per trace.
4. Audit search with an integrity check button.
5. Policy editor and simulator, approvals queue, alerts, revoke buttons.

Phase 8: SDK and Integrations (about 1 week)

1. Python SDK: register, get token, delegate, task context.
2. Adapters for LangChain or LangGraph and the Claude Agent SDK.
3. SIEM export (JSON lines or syslog).

Phase 9: Hardening, Demo, and Docs (1 week)

1. Load test the gateway and measure added latency.
2. Run the attack scenarios (section 14) and fix gaps.
3. Write architecture docs, the threat model, and a quickstart.
4. Record a 3-minute demo video.

13. Security Hardening Checklist

- Fail closed everywhere (policy, auth, revocation lookups).
- Short token TTLs (5 to 15 minutes) and unique `jti`.
- Bind tokens to the agent instance to limit theft value.
- Never log secrets, and redact tokens, keys, and sensitive arguments.
- Rotate signing keys and support multiple active keys through JWKS.
- Rate limit by agent, tool, and task.
- Validate tool arguments against schemas before policy checks.
- Protect the control plane with human authentication and MFA via an external IdP.
- Keep the downstream credential store separate and encrypted.
- Run dependency and container scanning in CI.

14. Demo and Attack Scenarios

Build a small set of scripted scenarios. These make the project convincing.

1. **Happy path**: an agent registers, gets a task, calls allowed tools, and every step appears in the audit trail.
2. **Stolen token**: replay an expired or another instance's token and watch it fail.
3. **Privilege escalation**: a sub-agent requests broader scope than its parent and is denied.
4. **Prompt-injection misuse**: a document tells the agent to email data to an external address. Policy requires approval and the human rejects it.
5. **Exfiltration burst**: an agent reads hundreds of records. Detection suspends it.
6. **Tampering**: an attacker edits an audit row and verification flags the break.
7. **Kill switch**: revoke a running agent mid-task and show immediate blocking.

15. How to Measure Success

- Percentage of tool calls with a complete audit record (target 100 percent).
- Gateway added latency (target: under 20 to 30 ms at p95 for allow decisions).
- Time from revoke to block (target: seconds).
- Attack scenarios blocked or detected (target: all of section 14).
- Policy coverage: share of tools with explicit policies.

16. Stretch Goals and Roadmap

- SPIFFE/SPIRE workload attestation and mTLS between components.
- Federation with enterprise IdPs (OIDC) for human principals.
- Agent-to-agent protocol support alongside MCP.
- Just-in-time access with automatic expiry and approval workflows.
- Risk scoring per agent and per action.
- Policy-as-code CI: test and review policies like application code.
- Multi-tenant mode and a Kubernetes operator.
- Compliance reports mapping controls to common frameworks.

17. Positioning as a Portfolio or Research Project

- Write a short paper or blog post: *"Why agents need their own identity layer"* using your threat model and attack results.
- Publish the repo with a clear README, architecture diagram, demo video, and a reproducible docker-compose demo.
- Highlight measurable results: latency, attacks blocked, audit integrity.
- Compare against existing approaches (static API keys, OAuth scopes alone, MCP without a gateway) to show the gap you fill.

18. Immediate Next Steps

1. Check that the name ANUBIS is free on GitHub and PyPI.
2. Decide the first protocol to support (recommended: MCP).
3. Create the repo and Docker Compose stack (Phase 0).
4. Build the registry and token service (Phase 1).

Once you have those, the data model and API endpoints in sections 7 and 10 can be turned directly into code, and I can draft the first FastAPI modules with you.